

Визуализация и анализ технологических зависимостей

Итерация 3

План на итерацию

1. Расширить источники данных: использовать DBPedia
2. Реализовать функционал для разворачивания графа в глубину
3. Реализовать функционал для отображения графов нескольких технологий и их сравнения
4. Подключить LLM-модель
5. Реализовать “RAG” для анализа полученного графа
6. Интегрировать результаты анализа в отчет
7. Интегрировать изображение графа в отчет
8. Исследование StackOverflow API для последующего внедрения

Результаты итерации: внедрение DBPedia

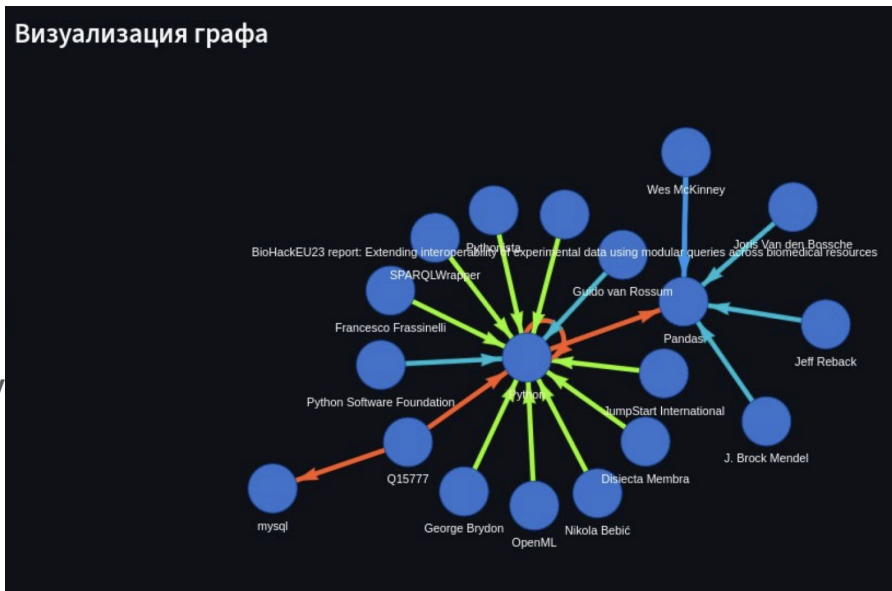
- Реализован дополнительный модуль сбора данных о технологиях DBPedia
- Новый модуль реализует существующие интерфейсы
- <https://github.com/moevm/mse1h2026-req-viz/pull/52/>

```
Результат парсинга:
{
  "nodes": [
    {
      "id": "Q15206305",
      "name": "docker",
      "type": "technology"
    },
    {
      "id": "Q96276695",
      "name": "Moby",
      "type": "entity"
    },
    {
      "id": "Q20971416",
      "name": "Solomon Hykes",
      "type": "entity"
    },
    {
      "id": "dbpedia:ARM_architecture_family",
      "name": "ARM architecture family",
      "type": "entity"
    },
    {
      "id": "dbpedia:Ppc64",
      "name": "Ppc64",
      "type": "entity"
    },
    {
      "id": "dbpedia:X86-64",
      "name": "X86-64",
      "type": "entity"
    },
    {
      "id": "dbpedia:Architecture",
      "name": "IBM mainframe expanded storage",
      "type": "entity"
    },
    {
      "id": "Q129730296",
      "name": "z/OS Container Extensions",
      "type": "entity"
    }
  ],
  "edges": [
    {
      "source": "Q107292855",
      "target": "Q16235208",
      "predicate": "creator",
      "source_id": "wikidata"
    },
    {
      "source": "Q4913907",
      "target": "Q16235208",
      "predicate": "uses",
      "source_id": "wikidata"
    },
    {
      "source": "Q138769927",
      "target": "Q16235208",
      "predicate": "used by",
      "source_id": "wikidata"
    },
    {
      "source": "Q180",
      "target": "Q16235208",
      "predicate": "used by",
      "source_id": "wikidata"
    },
    {
      "source": "Q107291842",
      "target": "Q16235208",
      "predicate": "used by",
      "source_id": "wikidata"
    },
    {
      "source": "Q115402046",
      "target": "Q16235208",
      "predicate": "used by",
      "source_id": "wikidata"
    },
    {
      "source": "Q489709",
      "target": "Q16235208",
      "predicate": "developer",
      "source_id": "wikidata"
    },
    {
      "source": "Q213660",
      "target": "Q16235208",
      "predicate": "developer",
      "source_id": "wikidata"
    },
    {
      "source": "Q251",
      "target": "Q16235208",
      "predicate": "programmed in",
      "source_id": "wikidata"
    },
    {
      "source": "Q460584",
      "target": "Q16235208",
      "predicate": "programmed in",
      "source_id": "wikidata"
    }
  ]
}
```

Пример работы модуля

Результаты итерации: развертка графа в глубину

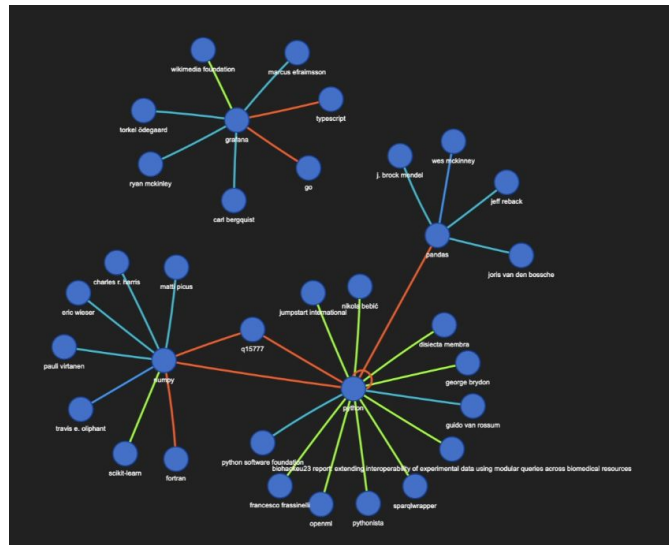
- Добавлена динамическая загрузка подграфа по клику на узел
- Объединены загруженные подграфы с текущей визуализацией
- Реализовано поведение переключения узлов (развернуть/свернуть по повторному клику)
- Обеспечена связность графа после сворачивания развернутых узлов.
- <https://github.com/moevm/mse1h2026-req-viz/pull/55>



Пример развертки графа

Результаты итерации: отображение графов нескольких технологий

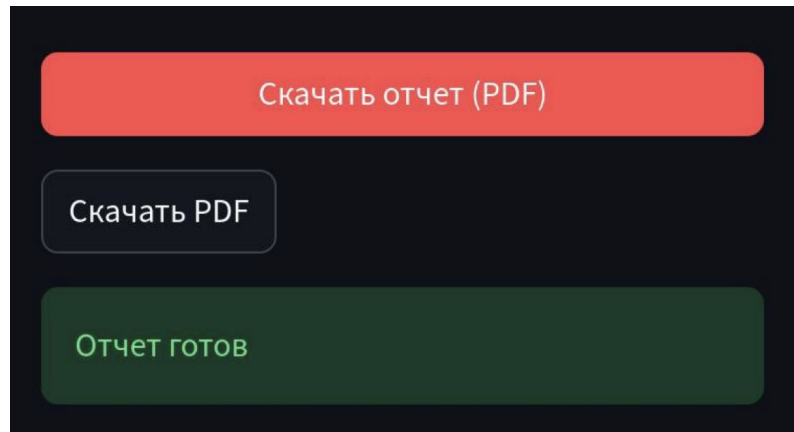
- Внедрен ThreadPoolExecutor для асинхронного выполнения I/O операций (Neo4j, парсер), чтобы не блокировать основной поток сервера
- Эндпоинт `/api/graph` теперь принимает список технологий.
- Реализована проверка существующих графов в БД (с учетом нескольких запрашиваемых технологий)
- Запрос отсутствующих данных с retry-логикой
- Объединение всех подграфов в один ответ с удалением дубликатов
- <https://github.com/moevm/mse1h2026-req-viz/pull/58>



Пример отображения нескольких графов

Результаты итерации: интеграция LLM-модели для анализа графа

- Добавлен эндпоинт, который по json представлению графа возвращает сгенерированный llm отчёт в формате .md
- Увеличен таймаут запросов к backend (из-за увеличения времени ответа при ожидании результатов генерации llm отчётов)
- Обеспечена возможность настройки промта для анализа нейросетью
- Добавлена инструкция по смене модели
- <https://github.com/moevm/mse1h2026-req-viz/pull/56>



Пример сообщения о готовности отчета

```
| 2026-05-03 10:20:33,777 [INFO] ecosystem_analyzer.main: Received graph data for report generation
| [GIN] 2026/05/03 - 10:20:55 | 200 | 21.636911716s | 10.219.0.4 | POST "/api/generate"
| 2026-05-03 10:20:55,417 [INFO] ecosystem_analyzer.main: Generated report successfully
| INFO: 10.219.0.5:58440 - "POST /api/report HTTP/1.1" 200 OK
```

Пример ответа от LLM на стороне backend

Результаты итерации: исследование StackOverflow API, интеграция ответа от LLM

- Исследована возможность использования StackOverflow API в качестве еще одного ресурса для получения данных для анализа.
- <https://github.com/moevm/ms-e1h2026-req-viz/pull/57>
- Ответ от LLM интегрирован в отчет, который выдается пользователю
- <https://github.com/moevm/ms-e1h2026-req-viz/pull/56>

Анализ: kafka

Дата: 03.05.2026 10:20:55

Дополнительная информация

Обзор технологии "kafka"

Краткое описание: Kafka - это open-source технология для распределенных систем, которая позволяет импортировать, экспортировать и обрабатывать streaming данных из различных источников в различные формы (например, CSV, JSON, Apache Log). Kafka используется в различных приложениях, включая обработку событий, управления контентом, аналитика и бизнес-аналитика.

Архитектурные зависимости:

- * Технологии: Apache Kafka (сервер), Apache ZooKeeper (сервер хранилища), Apache Flink (программируемое обработчик данных).
- * Компании: Netflix, Twitter, LinkedIn, Uber.
- * Персоны: Крейтон Роу, Даниэль Тейло, Алан Партон, J. K. Lin.

Экосистема и связи:

- * Создатель: LinkedIn.
- * Разработчик: Netflix, Twitter, Uber.
- * Инициатор: Apache Software Foundation.
- * Имилируемые риски: Невозможность обеспечения целостности данных в случае некомпимирования некомпликтных данных.

Потенциальные риски:

- * Невозможность обеспечения целостности данных в случае некомпликтных данных.
- * Могут быть проблемы с взаимным использованием ресурсов между different instances.
- * Возможность возникновения ошибок и дестанизации системы.

Рекомендации:

- * Используйте Kafka для обработки streaming данных.
- * Ограничьте количество источников и типов данных, которые могут быть одновременно связаны с одним каналом.
- * Используйте Kafka Connect для централизованного управления потоком данных.
- * Протвиорегируйте данные, чтобы избежать дублирования.
- * Оптимизируйте систему для повышения производительности.

Пример отчета, сгенерированного LLM

Результаты итерации: тестирование

- в рамках 2-ой итерации были подготовлены unit-тесты на модуль сохранения графов в neo4j
- на текущей итерации дополнительно реализованы интеграционные тесты, проверяющие: парсер, модуль сохранения в neo4j, backend, отправки HTTP-запроса на backend для сбора информации об:
 - одной технологии
 - нескольких технологий
- подготовленные тесты внедрены в GitHub Actions
- <https://github.com/moevm/mse1h2026-req-viz/pull/67>

integration-tests (3.14)

succeeded 2 hours ago in 5m 37s

- > ✓ Set up job
- > ✓ Checkout code
- > ✓ Set up Python 3.14
- > ✓ Install Poetry
- > ✓ Configure Poetry
- > ✓ Install dependencies (ecosystem_analyzer)
- > ✓ Install integration test dependencies
- > ✓ Set up Docker
- > ✓ Start test environment
- > ✓ Run integration tests
- > ✓ Cleanup test environment
- ⌚ Debug logs on failure
- > ✓ Post Set up Docker
- > ✓ Post Set up Python 3.14
- > ✓ Post Checkout code
- > ✓ Complete job

CI workflow для запуска тестов

План на следующую итерацию

1. Доработка парсера: параметр `rel_types` должен быть опциональным
2. Доработка парсера: возвращаются узлы с длинными названиями, либо с идентификаторами
3. Устранение недочета: БД иногда возвращает не все узлы
4. Доработать UI: при включении светлой темы, не видно имен узлов
5. Доработать UI: разбить исходный код на более маленькие модули, отвечающие за свою часть функционала
6. Доработать UI/парсер: изменить логику раскраски графа, чтобы цвет отношений задавался на основе данных от парсера
7. Интегрировать StackOverflow API для анализа количественных связей
8. При исследовании графа в глубину добавить строку статуса с состояниями: нет дополнительных узлов, ищу дополнительные узлы, и пр.
9. Аналогично при формировании отчета добавить обновляющуюся строку состояния генерации отчета
10. Добавить возможность отключения генерации отчета с помощью LLM (остается на усмотрение пользователя)